

Faculty of Engineering and Technology					
Ramaiah University of Applied Sciences					
Department		CSE/ISE/AIML/MC		Programme	B. Tech
Semester/Batch		VI / CSE			
Course Code		CSD303A		Course Title	Modern Application Development
Course Leader					
Assignment					
Register No.		23ETAI410035		Name of the Student:	Pratyush Pattanaik
Sections		Marking Scheme	Marks		
			Max Marks	First Examiner Marks	Moderator
	Part – 1	Infosys Springboard Full Stack Certification (Infosys Springboard)	15		
		Completion of Infosys Springboard Certification			
		Progressive Web Application Development			
		Certification Exam Score 35–59 marks			
		Certification Exam Score 60 and above			
	Part – 2	Design and develop a modern full stack web application implementing concepts from all 5 units.	15		
		Suggested Project Ideas - Toll road management - Real-time auction platform - Restful Blogging API - Health Tracker Application			
	Max Marks		30		

Course Marks Tabulation				
Component- CET B Assignment	First Examiner	Remarks	Second Examiner	Remarks
1				
2				
Total Marks				

Signature of First Examiner

Signature of Second Examiner

Please note:

1. Documental evidence for all the components/parts of the assessment such as the reports, photographs, laboratory exam / tool tests are required to be attached to the assignment report in a proper order.
2. The First Examiner is required to mark the comments in RED ink and the Second Examiner's comments should be in GREEN ink.
3. The marks for all the questions of the assignment have to be written only in the **Component – CET B: Assignment** table.
4. If the variation between the marks awarded by the first examiner and the second examiner lies within +/- 3 marks, then the marks allotted by the first examiner is considered to be final. If the variation is more than +/- 3 marks then both the examiners should resolve the issue in consultation with the Chairman BoE.

Assignment

Instructions to students:

1. The assignment has **two parts**:
Part 1: Infosys Springboard Full Stack Certification
Part 2: Development of a Full Stack Web Application
2. **Part 1 (15 Marks):**
Infosys Springboard Certification – Full Stack Development
 - Complete all modules – 9 Marks
 - Exam score 35–59 – 12 Marks
 - Exam score 60 and above – 15 Marks
3. **Part 2 (15 Marks):**
FULL STACK APPLICATION MINI PROJECT
 - i. Problem Statement (3 Marks)
 - ii. Frontend Implementation (3 Marks)
 - iii. Backend Development (3 Marks)
 - iv. Database Integration (3 Marks)
 - v. Deployment & Output (3 Marks)

COURSE OUTCOMES

CO Description

CO1. Apply modern front-end development frameworks to build responsive web applications.

CO2. Develop robust RESTful APIs and integrate with databases for backend solutions.

CO3. Deploy containerized applications using CI/CD pipelines on cloud platforms.

CO4. Build basic mobile and progressive web applications using modern tools.

CO5. Implement security measures and emerging technologies like AI and IoB in applications.

CONCLUSION

This assignment helped in understanding full stack development and applying theoretical concepts into real-world applications. It improved practical skills in frontend, backend, and database integration.

REFERENCES

- Infosys Springboard
- MDN Web Docs
- React Official Documentation
- Node.js Official Documentation

SIGNATURES

Signature of First Examiner:

Signature of Second Examiner:

Certification Link:

[Progressive Web Application Development - TOC | Infosys Springboard](#)

Submission Details

- **Submission Date:** 12-05-2026
- **Total Marks:** 30
- **Include:**
 - Part1, Infosys Springboard certification document (Part 1 evidence)
 - Part 2 Development of a Full Stack Web Application (Part 2 evidence- Live demo and report submission)
- **Late submissions will not be accepted.**
- The report must be **word-processed, neatly formatted**, and limited to **10 pages**.
- Ensure proper **citations and dataset references** are included.

Course Outcomes:

Part1: - Infosys Springboard certification document



COURSE COMPLETION CERTIFICATE

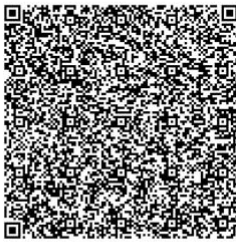
The certificate is awarded to

Pratyush Pattanaik

for successfully completing the course

Progressive Web Application Development

on May 11, 2026



Issued on: Thursday, May 14, 2026
To verify, scan the QR code at <https://verify.onwingspan.com>



Congratulations! You make us proud!


Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

Progressive Web Application Development



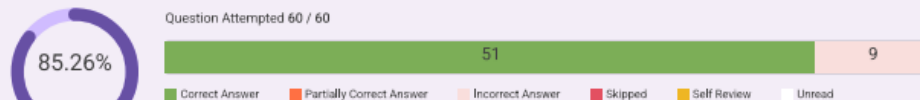
Congratulations

You have completed the assessment

Email : infosysheadstart_b51b617a-dd4e-4a83-ae92-db653327677c@iittonwingspan.com

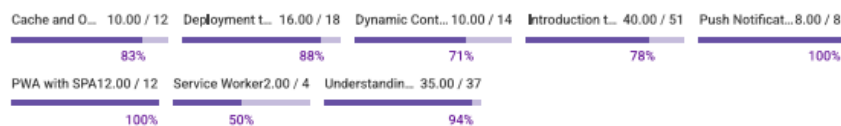
Total Duration: 01h 00m | Time Utilised: 31m 26s | Average Time Per Question: 31s | Assessment Attempts: 1/99

Question Attempted 60 / 60

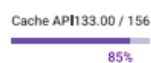


Beginner Questions	Intermediate Questions	Advanced Questions
45/45 Attempted 76.00/90 Your Score	9/9 Attempted 32.00/36 Your Score	6/6 Attempted 25.00/30 Your Score

Section Based Score



Skill Based Score



1. Part 2 :

PLANTCARE FULL STACK WEB APP

WEBSITE: - <https://plant-care-pwa-production.up.railway.app>

GitHub Repo: - <https://github.com/conuckrater14/plant-care-pwa>

Abstract

PlantCare is a full stack web application designed to help users manage their plants in a simple and organized way. It allows secure registration and login, plant storage, watering reminders, care history tracking, and health monitoring through a browser-based dashboard. The application uses React for the frontend, Node.js and Express for the backend, MongoDB for data storage, and JWT for authentication.

This project was developed to solve the common problem of forgetting plant care schedules and losing track of maintenance tasks. By combining a responsive interface with a secure backend and persistent database, PlantCare provides a practical solution for everyday plant management.

i. Introduction Features

In day-to-day life, many people keep indoor and outdoor plants but fail to maintain a proper care schedule. Important tasks such as watering, fertilizing, repotting, and monitoring plant health are often done manually or remembered informally. This creates confusion, inconsistency, and neglect, especially when users manage multiple plants with different care requirements. As a result, plants may dry out, become unhealthy, or receive improper care.

To solve this issue, a full stack web application named PlantCare was developed. The purpose of PlantCare is to provide a centralized digital platform where users can securely log in, add their plants, store care-related information, and monitor plant health in a simple and organized way.

The application is designed to support the following major functions:

- User registration and login.
- Personal dashboard to manage plants.
- Add, view, update, and delete plant records.
- Track watering schedule and plant care history.
- Display plant health status through score-based indicators.
- Store all information permanently in a database.
- Deploy the complete application online for real-time use.

ii. Frontend Implementation

The frontend of PlantCare was developed using **React**, which is a modern JavaScript library for building interactive user interfaces. React was selected because it supports component-based development, reusability, dynamic rendering, and efficient state handling.

Frontend Technologies and Packages Used

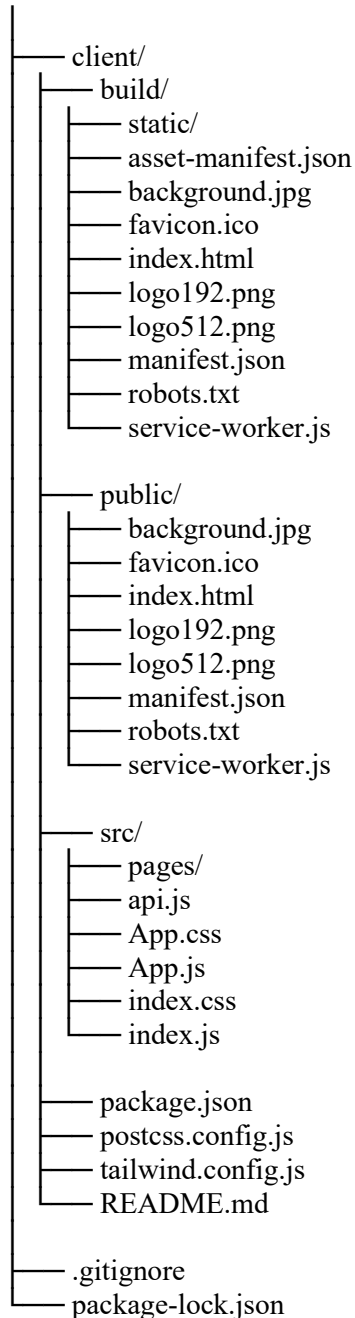
The following packages and tools were used in the frontend section of the project:

<i>Package / Tool</i>	<i>Purpose</i>
<i>React</i>	Core library used to build the user interface
<i>React DOM</i>	Used to render React components in the browser
<i>React Router DOM</i>	Used for page routing and navigation
<i>JavaScript Fetch API</i>	Used to communicate with backend API endpoints
<i>Tailwind CSS / utility styling</i>	Used for styling dashboard, forms, cards, buttons, and responsive layout
<i>Local Storage</i>	Used to store JWT token after login

Frontend File Structure

The frontend follows a modular structure. A typical organization of the client side is shown below:

PLANT-CARE-PWA/



Frontend Working

The App.js file acts as the root component of the application and manages the navigation between different pages such as login, dashboard, and plant detail view. The api.js file centralizes all frontend API calls, making it easier to manage communication with the server.

The main dashboard is implemented in Dashboard.js. This file displays all plants belonging to the logged-in user in a card layout. Each card shows plant name, species, health score, next watering date, and a visual water status bar. The dashboard also includes:

- Add Plant form.
- Delete plant option.
- Logout button.
- Loading state for data fetching.
- Dynamic count of total plants.
- Need-water indication for plants that require immediate care.

The user interface is interactive and responsive. Buttons, cards, form fields, and visual status badges provide a better user experience. The use of React state (useState) and side effects (useEffect) makes the application update dynamically when plants are added or removed.

The API helper file api.js contains functions such as:

- register(data)
- login(data)
- getPlants()
- createPlant(data)
- updatePlant(id, data)
- deletePlant(id)

These functions use the fetch() API and attach the JWT token through the Authorization header for protected requests.

iii.Backend Development

The backend of PlantCare was developed using Node.js and Express.js. The backend is responsible for authentication, routing, business logic, protected API access, and communication with the MongoDB database.

Backend Technologies and Packages Used

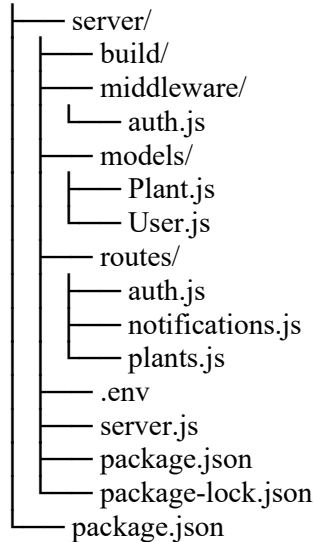
The following packages were used in the backend:

<i>Package</i>	<i>Purpose</i>
<i>Express</i>	Used to create the web server and API routes
<i>Mongoose</i>	Used to define schemas and interact with MongoDB
<i>JSON Web Token (jsonwebtoken)</i>	Used to generate and verify authentication tokens
<i>bcryptjs</i>	Used to hash passwords securely
<i>dotenv</i>	Used to load environment variables
<i>cors</i>	Used to enable communication between frontend and backend
<i>dns</i>	Used in server configuration

Backend File Structure

The backend is structured into separate folders for routes, models, middleware, and deployment-related files.

PLANT-CARE-PWA/



Server Setup

The server.js file initializes the Express application, loads environment variables, registers middleware, connects to MongoDB, and starts the server. It also defines the main route mapping:

- /api/auth → Authentication routes
- /api/plants → Plant management routes
- /api/notifications → Notification-related routes[file:2490]

The backend uses express.json() to parse incoming JSON request bodies and cors() to allow frontend-backend interaction.

Authentication Module

The auth.js route file handles user registration and login. It contains:

- POST /register — Creates a new user after checking whether the email already exists.
- POST /login — Validates email and password, then returns a JWT token if the credentials are correct.

During registration and login, the server responds with:

- JWT token
- User ID
- User name
- User email[file:2480]

This token is later stored in the frontend and used for accessing protected plant routes.

Plant Management Module

The plants.js route file implements the main plant-related backend operations. The available routes are:

<i>Method</i>	<i>Endpoint</i>	<i>Purpose</i>
GET	/api/plants	Retrieve all plants for logged-in user
GET	/api/plants/:id	Retrieve single plant details
POST	/api/plants	Create new plant
PUT	/api/plants/:id	Update plant details
DELETE	/api/plants/:id	Delete a plant
POST	/api/plants/:id/water	Log watering action and update care history

All these routes are protected using the authentication middleware, which ensures that users access only their own plant data.

iv.Database Integration

The application uses **MongoDB** as the database and **Mongoose** for schema creation and model-based operations. Database integration is one of the core parts of this project because all user and plant information is stored permanently and retrieved dynamically.

User Model

The User.js file defines the schema for storing user information. The main fields in the schema are:

- name
- email
- password
- pushSubscription
- automatic timestamps (createdAt, updatedAt)

The password is not stored directly. Instead, a Mongoose pre-save hook hashes it using `bcrypt.hash()` before saving it into the database. During login, the `comparePassword()` method compares the entered password with the hashed one stored in the database.

Plant Model

The Plant.js file defines the structure of plant records. Important fields in this model include:

- owner — references the logged-in user.
- name — name of the plant.
- species — plant species.
- photo — image URL or photo path.
- wateringFrequency — number of days between watering cycles.
- lastWatered — last watering date.
- nextWatering — automatically calculated upcoming watering date.
- fertilizerFrequency — fertilization cycle.
- lastFertilized — previous fertilizer date.
- notes — additional information about the plant.
- healthScore — numeric health measure.
- careHistory — embedded sub-document array for care tracking.

The schema also contains a `careHistorySchema`, which stores details of each care action such as:

- action type (watered, fertilized, repotted, pruned)
- date
- note[file:2486]

A pre-save hook computes the next `Watering` date automatically based on `lastWatered` and `wateringFrequency`, reducing manual work and maintaining data consistency.

Database Operations

The backend performs several Mongoose operations to interact with the database:

- `findOne()` to locate a user or plant.
- `find()` to list all plants belonging to a user.
- `create()` to insert new user or plant records.
- `findOneAndUpdate()` to update plant information.
- `findOneAndDelete()` to remove a plant.
- `save()` to persist changes after watering events.

Thus, database integration is fully implemented and connected with both frontend forms and backend routes.

v.Deployment & Output

After development and testing in the local environment, the application was deployed on Railway so that it could run as a real-world cloud-hosted project. Railway was used because it provides easy deployment for Node.js applications and supports environment variable configuration, GitHub integration, and public hosting.

Deployment Process

The deployment process involved the following steps:

1. Build the React frontend into production-ready static files.
2. Place the React build inside the server deployment structure.
3. Configure Express to serve the frontend build files.
4. Configure API calls to use the same `/api` path in production.
5. Push the updated project to GitHub.
6. Connect the repository to Railway.
7. Add environment variables such as `MONGO_URI`, `JWT_SECRET`, and deployment port.

Final Production Setup

In the final working deployment:

- The Express backend runs on Railway.
- The React frontend is served by Express from the build folder.
- API routes are available under `/api`.
- The frontend and backend run under the same origin, solving routing and production communication issues.

Output of the Application

The final output of the application includes the following successfully working modules:

- Login and registration system.
- Secure token-based authentication.
- Dashboard showing all plants.
- Add new plant feature.
- Delete plant feature.
- Health and watering status indicators.
- Plant detail and plant care tracking.
- End-to-end working deployment on Railway.

The final working version of the project was also committed to GitHub as a recovery point after confirming that the deployment was successful.

vi. Website: -

The final output of the PlantCare application consists of a working user interface connected to a secure backend and database. The deployed application supports registration, login, plant management, watering tracking, and plant health monitoring through a browser-based dashboard.

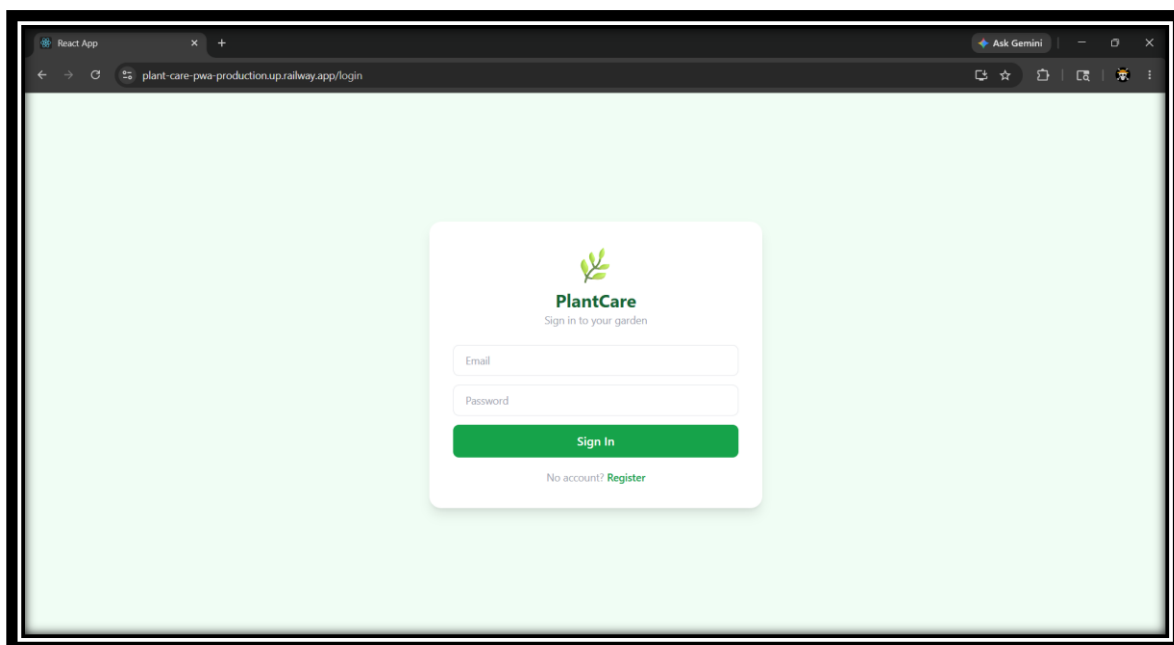


Figure 1:- Login/Register Page

This screen allows a new user to create an account and an existing user to log in using email and password. Authentication is handled through backend routes that return a JWT token after successful login or registration.

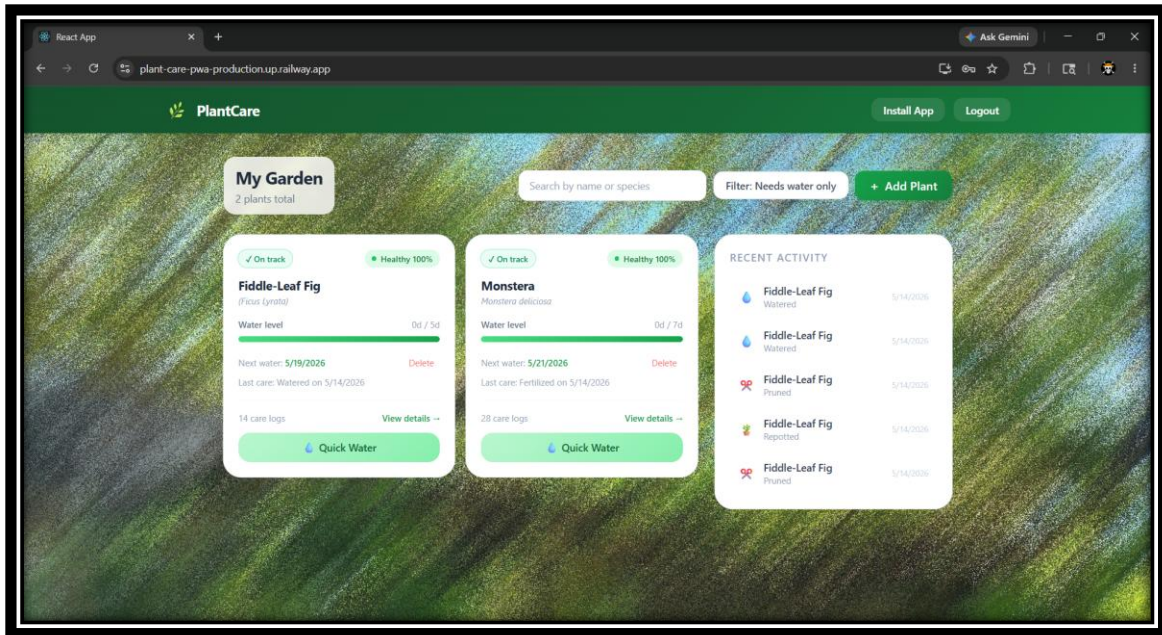


Figure 2: - Dashboard Page

This is the main page of the application where the user can view all plants, total plant count, and watering alerts. The dashboard also shows plant cards with health badges and watering indicators. Each plant is shown in a card layout with details such as species, health score, and next watering date. This view improves readability and helps users quickly identify which plants need attention.

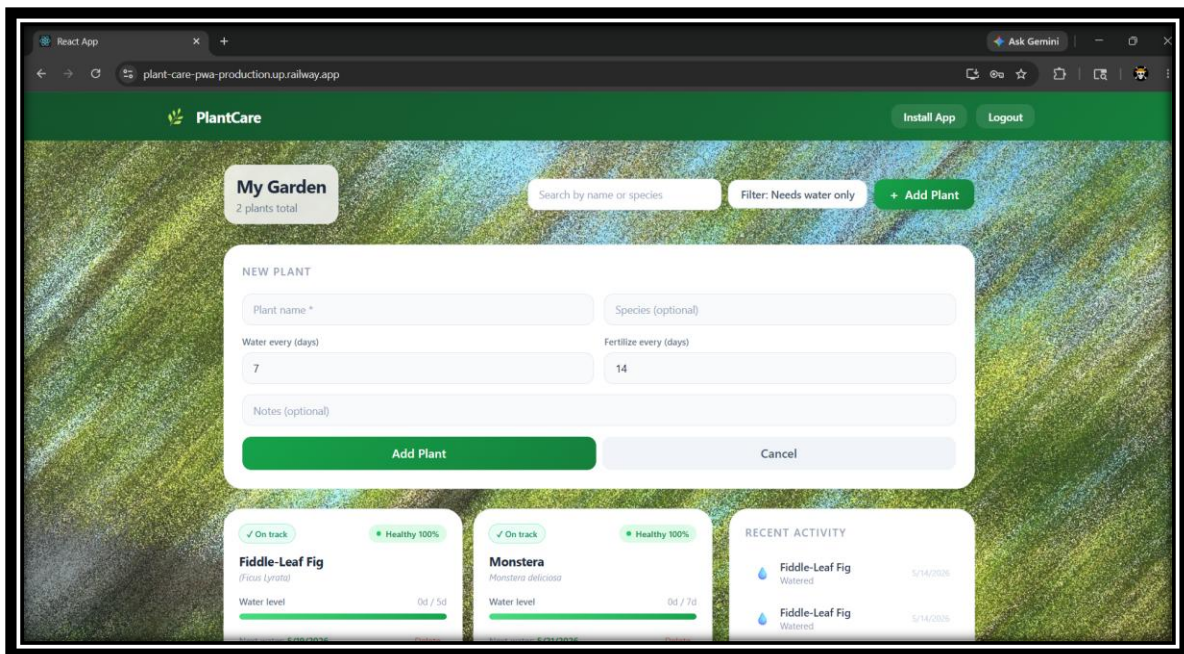


Figure 3: - Adding Plants Form

This screen or popup form allows users to enter plant name, species, watering frequency, fertilizer frequency, and notes. The entered data is sent to the backend and stored in the database.

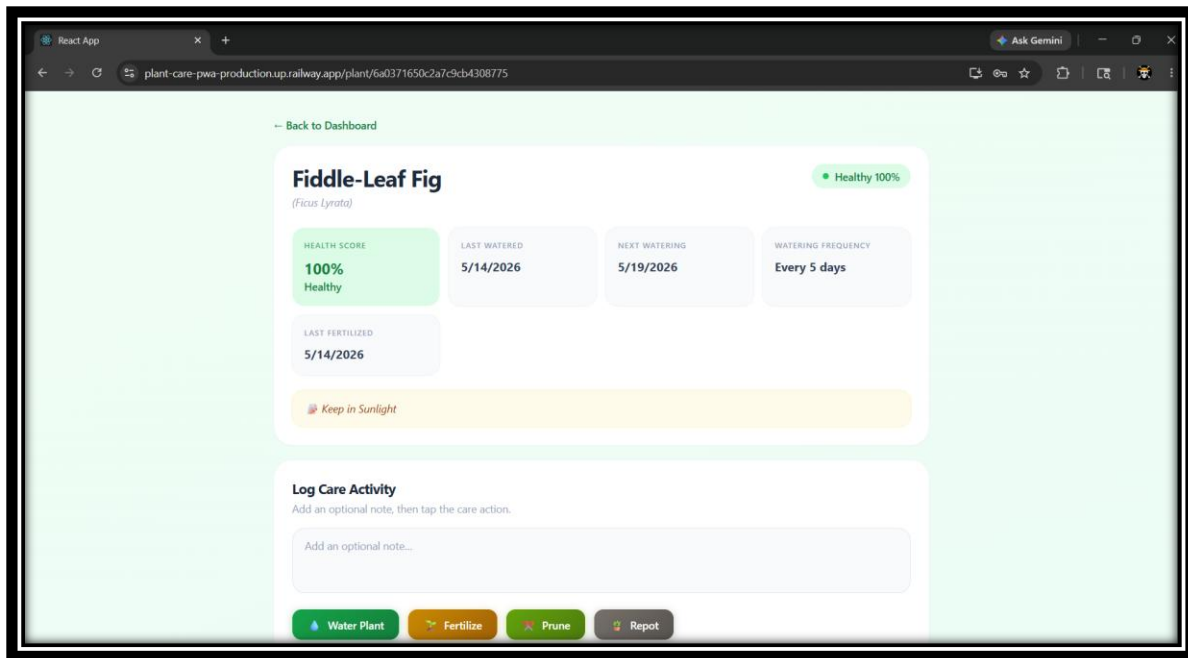


Figure 4: - Plant Detail Page

This screen displays the full details of a selected plant and supports plant-specific care actions such as watering and viewing care history.

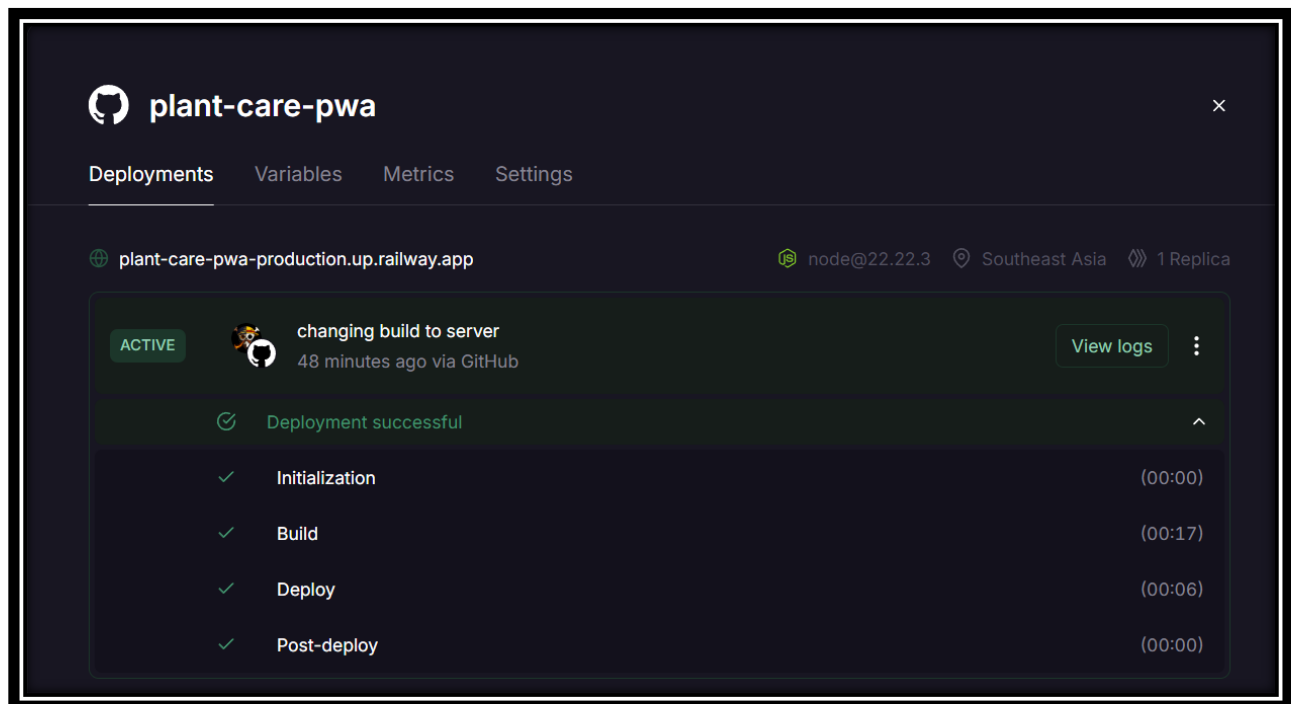


Figure 5: - Railway Build Output

This serves as proof that the application was successfully deployed and tested online.

Code Snippets: -

Sample Code 1: Authentication Route

The following backend code handles user registration and login. It checks existing users, validates credentials, and returns a JWT token for authenticated access.

```
server > routes > JS auth.js > ...
You, yesterday | 1 author (You)
1  const express = require('express');
2  const jwt = require('jsonwebtoken'); 55.3k (gzipped: 16.3k)
3  const User = require('../models/User');
4  const router = express.Router();
5
6  router.post('/register', async (req, res) => {
7    try {
8      const { name, email, password } = req.body;
9      const existing = await User.findOne({ email });
10     if (existing) return res.status(400).json({ message: 'Email already exists' });
11     const user = await User.create({ name, email, password });
12     const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
13     res.status(201).json({ token, user: { id: user._id, name: user.name, email: user.email } });
14   } catch (err) {
15     res.status(500).json({ message: err.message });
16   }
17 });
```

Sample Code 2: Plant Management Route

This backend route fetches all plants belonging to the logged-in user and sorts them by next watering date. It is protected using authentication middleware.

```
server > routes > JS plants.js > ...
You, yesterday | 1 author (You)
1  const express = require('express');
2  const Plant = require('../models/Plant');
3  const auth = require('../middleware/auth');
4  const router = express.Router();
5
6  router.get('/', auth, async (req, res) => {
7    try {
8      const plants = await Plant.find({ owner: req.user.id }).sort({ nextWatering: 1 });
9      res.json(plants);
10   } catch (err) {
11     res.status(500).json({ message: err.message });
12   }
13 });
14
15 router.get('/:id', auth, async (req, res) => {
16   try {
17     const plant = await Plant.findOne({ _id: req.params.id, owner: req.user.id });
18     if (!plant) return res.status(404).json({ message: 'Plant not found' });
19     res.json(plant);
20   } catch (err) {
21     res.status(500).json({ message: err.message });
22   }
23 });
24
```


Sample Code 3: User Schema

The User schema stores account details and hashes passwords before saving them to the database using bcrypt.

```
server > models > JS Userjs > ...
You, yesterday | 1 author (You)
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs'); 20.6k (gzipped: 9.2k)
3
4  const userSchema = new mongoose.Schema({
5    name: { type: String, required: true },
6    email: { type: String, required: true, unique: true },
7    password: { type: String, required: true },
8    pushSubscription: { type: Object, default: null },
9  }, { timestamps: true });
10
11 userSchema.pre('save', async function() {
12   if (!this.isModified('password')) return;
13   this.password = await bcrypt.hash(this.password, 12);
14 });
15 userSchema.methods.comparePassword = async function(candidatePassword) {
16   return await bcrypt.compare(candidatePassword, this.password);
17 };
18
19 module.exports = mongoose.model('User', userSchema);
```

Sample Code 4: Plant Schema

The Plant schema stores plant details, care history, and watering schedule information.

```
server > models > JS Plantjs > ...
You, yesterday | 1 author (You)
1  const mongoose = require('mongoose');
2
3  const careHistorySchema = new mongoose.Schema({
4    type: { type: String, enum: ['watered', 'fertilized', 'repotted', 'pruned'] },
5    date: { type: Date, default: Date.now },
6    note: { type: String, default: '' }
7  });
8
9  const plantSchema = new mongoose.Schema({
10   owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
11   name: { type: String, required: true },
12   species: { type: String, default: '' },
13   photo: { type: String, default: '' },
14   wateringFrequency: { type: Number, default: 3 },
15   lastWatered: { type: Date, default: Date.now },
16   nextWatering: { type: Date },
17   fertilizerFrequency: { type: Number, default: 14 },
18   lastFertilized: { type: Date, default: Date.now },
19   notes: { type: String, default: '' },
20   healthScore: { type: Number, default: 100 },
21   careHistory: [careHistorySchema]
22 }, { timestamps: true });
23
24 plantSchema.pre('save', function() {
25   const now = new Date();
26   this.nextWatering = new Date(
27     (this.lastWatered || now).getTime() + this.wateringFrequency * 24 * 60 * 60 * 1000
28   );
29 });
30 module.exports = mongoose.model('Plant', plantSchema);
```

Sample Code 5: Frontend API Integration

The frontend communicates with the backend using functions defined in api.js. These functions include the JWT token in the request header for protected routes.

```

client > src > JS api.js > ...
  You, 11 hours ago | 1 author (You)
1  const BASE_URL = '/api';
2  const getToken = () => localStorage.getItem('token');
3
4  const headers = () => ({
5    'Content-Type': 'application/json',
6    Authorization: `Bearer ${getToken()}`,
7  });
8
9  const handleResponse = async (response) => {
10   const data = await response.json();
11   if (!response.ok) {
12     throw new Error(data.message || 'Something went wrong');
13   }
14   return data;
15 };
16
  You, 22 hours ago • fixed client

```

Sample Code 6: Dashboard Logic

The dashboard fetches plant data from the backend and updates the UI dynamically using React hooks.

```

client > src > pages > JS Dashboard.js > Dashboard
1  import { useState, useEffect, useMemo, useCallback } from 'react';
2  import { useNavigate } from 'react-router-dom';
3  import { getPlants, createPlant, deletePlant, waterPlant } from '../api';
4
5  function HealthBadge({ score = 0 }) {
6    const cfg =
7      score >= 80
8        ? { bg: '#dcfce7', text: '#15803d', dot: '#22c55e' }
9        : score >= 50
10         ? { bg: '#fef9c3', text: '#a16207', dot: '#eab308' }
11         : { bg: '#fee2e2', text: '#b91c1c', dot: '#ef4444' };
12
13    const label = score >= 80 ? 'Healthy' : score >= 50 ? 'Fair' : 'Critical';
14

```

vii.Conclusion

PlantCare is a practical full stack web application that solves a real everyday problem: helping users manage plant care in one organized place. Instead of relying on memory or scattered notes, the application gives users a secure system to register, log in, add plants, record care activities, and keep track of watering needs and plant health from a single dashboard. The project makes plant maintenance easier, more consistent, and more reliable.

The frontend was built to be simple, responsive, and easy to use, while the backend handles authentication, protected access, and plant management operations. The database stores user and plant information permanently, including care history and watering schedules, so the app can keep updating plant status accurately. This combination of frontend, backend, and database creates a complete working system rather than just a static interface.

One of the strongest parts of the project is the secure login system. Passwords are hashed before storage, and JWT tokens are used to protect user-specific routes, which means each user can access only their own plant data. This makes the application more realistic and closer to a production-level web app.

The deployment on Railway shows that the project is not limited to local development. It runs online as a live application, with the backend serving the frontend and API together from the same deployment. That final setup proves that the project works end to end and can be used in a real environment.

Overall, PlantCare demonstrates how modern web technologies can be combined to build a useful, secure, and deployable application. It reflects strong understanding of full stack development, database integration, authentication, and deployment workflows.

viii.References

1. React Documentation. Available at: <https://react.dev>.
2. React Learn. Available at: <https://react.dev/learn>.
3. Express.js Documentation. Available at: <https://expressjs.com>.
4. Express “Hello World” Example. Available at: <https://expressjs.com/en/starter/hello-world.html>.
5. Mongoose Documentation. Available at: <https://mongoosejs.com>.
6. Mongoose Get Started Guide. Available at: <https://mongoosejs.com/docs/>.
7. MongoDB Mongoose Integration Guide. Available at: <https://www.mongodb.com/docs/drivers/node/current/integrations/mongoose/mongoose-get-started/>.
8. MDN Web Docs: Express Node.js Guide. Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs

ix.Tools Used

The project was developed using the following tools and technologies:

1. React: - for building the frontend user interface.
2. React Router DOM: - for page navigation and route handling.
3. JavaScript Fetch API: - for connecting the frontend to backend APIs.
4. Node.js: - for running the server-side application.
5. Express.js: - for creating RESTful API routes and backend logic.
6. MongoDB: - for storing user and plant data.
7. Mongoose: - for defining database schemas and interacting with MongoDB.
8. Bcryptjs: - for password hashing and verification.
9. Jsonwebtoken: - for token-based authentication.
10. Dotenv: - for managing environment variables.
11. CORS: - for enabling frontend-backend communication.
12. Railway: - for deployment.
13. GitHub: - for version control and maintaining the project repository.